

Spring MVC — 复习要点与考点总结

Spring 框架与 Spring MVC 简介 (Framework & Introduction)

Spring Framework 概述 (Overview)

- **Spring Framework** 是一个为 Java 应用提供全面基础设施支持 (Infrastructure Support) 的 Java 平台。[p.4]
- 核心理念: 让开发者专注于业务应用, Spring 处理底层基础设施 (Handles the infrastructure so you can focus on your application)。[p.4]
- 支持通过 **POJO (Plain Old Java Object)** 构建应用, 非侵入式 (Non-invasively) 地应用企业级服务 (Enterprise Services)。[p.4]
- 适用于 Java SE 编程模型以及全部或部分 Java EE 场景。[p.4]
- 官网: <https://spring.io>。[p.5]
- 创始人 **Rod Johnson**: Interface21 公司 CEO, 丰富的 C/C++ 开发和金融行业业务背景; 1996 年开始关注 Java 服务器端技术, Servlet 2.4 和 JDO 2.0 专家组成员。[p.7]
- 2002 年著作 *Expert one-on-one J2EE Development without EJB*。[p.7]
- 核心信条: **Don't Reinvent the Wheel** (不要重复造轮子)。[p.7]

IoC 与 AOP 基础回顾 (IoC & AOP Review)

- **IoC (Inverse of Control, 控制反转)** / **Bean 容器**: 管理对象生命周期与依赖注入 (Dependency Injection, DI)。[p.2]
- **AOP (Aspect-Oriented Programming, 面向切面编程)**: 处理横切关注点 (Cross-cutting Concerns): 权限/安全管理切面、日志管理切面、事务管理切面。[p.2]
- 经典三层架构 (Three-tier Architecture): [p.2]
 - 表示层 (Presentation Layer): Servlet / JSP
 - 业务逻辑层 (Business Logic Layer): Service / Business Bean
 - 模型层/数据访问层 (Model/DAO Layer): Bean / DAO
- Spring MVC 运行于 IoC 容器之上, 所有对象 (Controller、Service 等) 均由 IoC 容器管理。[p.8]

Spring MVC 简介 (Introduction to Spring MVC)

- **Spring Web MVC** 是 Spring 体系中的轻量级 Web 表示层 (Presentation Layer) 框架。[p.8]
- Spring MVC 的核心是 **Controller (控制器)**, 用于处理请求和响应。[p.8]
- Spring MVC 基于 Spring IoC 容器运行, 所有对象被 IoC 容器管理。[p.8]
- 版本演进: [p.8]
 - **Spring 5.x**: 要求 JDK 8、Servlet 3.1 (Tomcat 8.5+); 支持 JDK 8+ 新特性; 支持 **响应式编程 (Reactive Programming)** — 基于事件回调机制。
 - **Spring 6.x**: 要求 JDK 17、Servlet 5.0 (Tomcat 10.x); 全面拥抱 Jakarta EE 命名空间。
- Maven 核心依赖: `spring-webmvc`。[p.11]

Spring MVC 架构与核心原理 (Architecture & Core Principles)

DispatcherServlet 请求处理流程 (Request Processing Flow)

Spring MVC 的核心是一个 前端控制器 (Front Controller) 模式，由 DispatcherServlet (前端调度器) 统一调度所有请求。完整处理流程如下：[p.10]

- ① 请求到达前端控制器：客户端 HTTP 请求首先到达 DispatcherServlet (前端控制器/Front Controller)，DispatcherServlet 是整个流程的总调度中心。它会委托 (Delegate) 给具体的控制器处理请求。[p.10]
- ② 处理器映射查询：前端控制器通过查询 Handler Mapping (处理器映射)，找到处理该 URL 请求的对应 Controller 及方法。HandlerMapping 维护了 URL 到 Handler 的映射关系表。[p.10]
- ③ 控制器处理请求：匹配到的 Controller 处理请求，包括解析请求参数、处理业务数据、调用业务逻辑层 (Service Layer) 等。Controller 是真正的请求处理单元。[p.10]
- ④ 返回模型与逻辑视图名：Controller 处理完毕，将模型数据 (Model) 打包，连同 逻辑视图名 (Logical View Name) 一并返回给前端控制器 DispatcherServlet。模型数据是一个 Map 类型的键值对集合。[p.10]
- ⑤ 视图解析：前端控制器调用 View Resolver (视图解析器)，将逻辑视图名 (如 "home") 解析为具体的视图实现 (如 /WEB-INF/views/home.jsp)。[p.10]
- ⑥ 视图渲染：获得具体的视图对象后，View (视图) 利用模型数据进行页面渲染 (Rendering)，将模型数据填充到视图模板中。[p.10]
- ⑦ 交付 Web 响应：渲染完成后，将最终的 HTML 内容作为 Web 响应交付给客户端。[p.10]

核心组件 (Core Components)

- **DispatcherServlet (前端控制器/前端调度器)**：Spring MVC 的统一入口，所有 HTTP 请求首先到达此处；协调各组件协同工作；可在 web.xml 中配置，也可通过 Java Config 方式配置。[p.10, 12]
- **Handler Mapping (处理器映射器)**：根据请求 URL、请求方法等信息找到对应的 Controller 及方法；Spring MVC 内置多种 HandlerMapping 实现 (如 RequestMappingHandlerMapping)。[p.10]
- **Handler Adapter (处理器适配器)**：负责调用具体的 Handler (Controller) 方法；将请求参数适配为方法参数；将方法返回值适配为 ModelAndView。[p.10]
- **Controller (控制器)**：处理具体的业务逻辑；解析请求参数、调用 Service、返回 Model 和逻辑视图名。Spring MVC 无需继承任何基类，通过注解驱动。[p.10, 14]
- **View Resolver (视图解析器)**：将 Controller 返回的逻辑视图名解析为具体的 View 对象；常配合前缀 (prefix) 和后缀 (suffix) 配置，如 /WEB-INF/views/ + viewName + .jsp。[p.10]
- **View (视图)**：负责将模型数据渲染为最终响应内容；支持多种视图技术 (JSP、FreeMarker、Thymeleaf 等)。[p.10]

与原生 Servlet 对比 (Comparison with Native Servlet)

- **原生 Servlet 开发**：需要继承 HttpServlet 基类；在 web.xml 中逐个配置 Servlet 和 URL 映射 (Servlet Mapping)；手动解析请求参数、手动进行类型转换；手动设置响应类型和输出内容；代码量大、重复度高、耦合紧密。[p.15, 17]
- **Spring MVC 开发**：通过注解 (@Controller, @GetMapping 等) 声明式配置，无需继承基类；URL 映射粒度更细 (方法级)；自动封装请求参数并完成类型转换；自动处理响应内容类型和输出；开发效率大幅提升。[p.14-15, 17]

URL 映射 (URL Mapping)

URL 映射原理与特点 (Principles & Characteristics)

- Spring MVC 通过 **URL 映射 (URL Mapping)** 将 Web 请求的 URL 路径与 Controller 中的方法进行映射绑定。[p.19]
- URL 映射基于 **方法级别 (Method-level Mapping)**，比传统 web.xml 中 Servlet 级别的映射粒度更小，使用更灵活。[p.19]
- 一个 Controller 类中可以定义多个方法，每个方法映射到不同的 URL 和处理不同的 HTTP 请求方法。[p.19]

URL 映射注解参考表 (Annotation Reference Table)

表 1: URL 映射注解参考表 [p.14, 19]

注解 (Annotation)	作用 (Purpose)	使用说明 (Usage)
@RequestMapping	通用映射，不区分请求方法	常用于 Controller 类级别，设置全局路径前缀；也可用于方法级别，通过 <code>method</code> 属性指定请求方式
@GetMapping	GET 请求映射	作用于具体方法，处理 HTTP GET 请求；对应查询操作
@PostMapping	POST 请求映射	作用于具体方法，处理 HTTP POST 请求；对应新增操作
@PutMapping	PUT 请求映射	作用于具体方法，处理 HTTP PUT 请求；对应更新操作
@DeleteMapping	DELETE 请求映射	作用于具体方法，处理 HTTP DELETE 请求；对应删除操作
@PatchMapping	PATCH 请求映射	作用于具体方法，处理 HTTP PATCH 请求；对应部分更新操作

URL 映射使用范例 (Usage Examples)

- 类级别映射: `@RequestMapping("/user")` 标注在 Controller 类上，则该类所有方法 URL 均以 `/user` 为前缀。[p.19]
- 方法级别映射: `@GetMapping("/hello")` 标注在方法上，完整访问 URL 为 `http://localhost:8080/hello`。[p.14, 19]
- 组合使用: 类上 `@RequestMapping("/api")` + 方法上 `@PostMapping("/login")`，完整映射为 `POST /api/login`。[p.19]
- RESTful 路径变量: `@GetMapping("/user/{id}")` 配合 `@PathVariable` 注解获取路径中的值。[p.21]

请求参数获取 (Request Parameter Binding)

参数获取原理 (Acquisition Principle)

- 模型驱动 (Model-Driven): Controller 对象构造时，Spring MVC 自动构建一个 **Model 对象 (Map 类型)**，用于存放请求相关的值，替代传统 `request.setAttribute()` 的方式。[p.21]
- 请求到达 Controller 前会经过一系列 **过滤器/拦截器 (Filters/Interceptors)**，实现类型转换 (Type Conversion)、集合取值等功能，开发者可对其进行扩展 (Extension)。[p.21]
- 参数绑定过程: HTTP 请求参数 → Spring MVC 解析 → 类型转换 → 绑定到方法参数或 JavaBean。

[p.21]

三种参数获取方式 (Three Parameter Binding Methods)

- 方式一：方法参数直接接收 (**Direct Parameter Binding**)。[p.21–23]
 - 基于名称匹配规则，Spring MVC 自动将同名请求参数绑定到方法参数。
 - 要求前后端属性名称一致 (Name Convention)。[p.22]
 - 若属性名称不一致，使用 `@RequestParam` 注解进行显示映射：
`@RequestParam("pageNum") int page。` [p.23]
 - `@RequestParam` 支持设置默认值 `defaultValue` 和是否必须 `required`。[p.23]
- 方式二：JavaBean 封装接收 (**JavaBean Binding**)。[p.21, 24]
 - 使用 JavaBean (POJO) 接收封装后的大量数据，更为常用。
 - 配合 `@RequestBody` 注解，将 HTTP 请求体 (Request Body) 自动转换为 Java 对象 (常用于 JSON 数据接收)。
 - Spring MVC 自动调用 Setter 方法进行属性赋值，支持级联属性 (如 `user.address.city`)。
- 方式三：路径变量与请求头 (**URI Path & Header Values**)。[p.21]
 - `@PathVariable`：获取请求 URI 模板中的变量值，用于 RESTful 风格 URL。
 - `@RequestHeader`：获取 HTTP 请求头 (Header) 中的值，如 `User-Agent`、`Content-Type` 等。

参数绑定注解对比表 (Parameter Binding Comparison)

表 2: 参数绑定注解对比 [p.21, 23, 24, 29]

注解	数据来源	适用场景	关键属性
<code>@RequestParam</code>	URL 查询参数或表单数据	单个参数获取，前后端名称不一致时显式映射	<code>name/value</code> , <code>required</code> , <code>defaultValue</code>
<code>@RequestBody</code>	HTTP 请求体 (Body)	JSON/XML 格式数据绑定到 JavaBean, POST/PUT 请求	<code>required</code>
<code>@PathVariable</code>	URI 路径变量	RESTful URL 参数提取	<code>name/value</code> , <code>required</code>
<code>@RequestHeader</code>	HTTP 请求头	获取 Header 信息 (Token, User-Agent 等)	<code>name/value</code> , <code>required</code> , <code>defaultValue</code>
<code>@ModelAttribute</code>	请求参数 + Model 数据	数据预初始化、自定义取值赋值方法、表单回填	<code>name/value</code> , <code>binding</code>

参数类型转换 (Type Conversion)

- Spring MVC 内置了丰富的类型转换器 (Converter)，自动将字符串参数转换为目标类型 (`int`, `long`, `double`, `Date` 等)。[p.21]
- 可自定义类型转换器，实现 `Converter<S, T>` 接口并注册到 Spring MVC 的格式化注册器 (FormatterRegistry)。[p.21]
- 格式化注解：`@DateTimeFormat` (日期格式)、`@NumberFormat` (数字格式) 可与参数绑定配合使用。[p.21]

响应处理与视图 (Response Handling & Views)

响应处理方式 (Response Methods)

- Spring MVC 对传统 Servlet 的响应处理进行了大量简化, 通过 **ModelAndView** 对象将响应的内容 (Model) 和视图 (View) 进行解耦合 (Decoupling)。[p.25]
- **@ResponseBody**: 不进行页面跳转和视图渲染, 直接输出响应文本; 将方法返回值序列化为指定格式 (JSON、XML 等) 写入 HTTP 响应体; 实际开发中一般返回 JSON 字符串。[p.25-26]
- **@RestController**: 类级别注解, 等同于 **@Controller** + **@ResponseBody** 组合; 标注后该类所有方法返回值均直接写入响应体, 适用于 RESTful API 开发。[p.14, 25]
- **ModelAndView**: 同时携带模型数据 (Model) 和视图名 (View Name); 通过 JSP、FreeMarker、Thymeleaf 等模板引擎进行视图渲染。[p.25]
- 默认视图采用 JSP 方式; Spring 老版本推荐 FreeMarker; Spring 3.x 开始推荐 Thymeleaf。[p.25, 30]

Model / ModelMap / ModelAndView 对象对比 (Data Container Objects Comparison)

- Spring MVC 中用于存放数据的三个核心对象: **ModelMap**、**Model**、**ModelAndView**。[p.29]
- **ModelMap**: 由拦截器自动创建 (类似于 Struts 2 的值栈概念), 在 Controller 的方法之前运行; 基本的 Map 类型数据容器。[p.29]
- **Model**: 同样由拦截器自动创建, 继承自 ModelMap, 提供了更丰富的方法; 是 ModelMap 的增强版。[p.29]
- **ModelAndView**: 在 Model 基础上添加了视图对象 (View); 需要程序员手动创建 (`new ModelAndView()`); 同时包含模型数据和视图跳转信息。[p.29]
- 可使用 **@ModelAttribute** 注解自定义取值及赋值方法, 常用于 Controller 中每个方法执行前的公共数据初始化。[p.29]
- 根据继承关系, 赋值和取值能力排序: **ModelAndView** > **Model** > **ModelMap**。[p.29]

页面跳转方式 (Forward vs Redirect)

- **Forward (转发)**: Spring MVC 中 ModelAndView 的默认跳转方式。[p.28]
 - 服务器内部跳转, URL 地址栏不变。
 - 请求对象 (HttpServletRequest) 在跳转过程中共享, Model 数据存放于 request 作用域。
 - 适用于同一应用内页面跳转, 效率较高。
- **Redirect (重定向)**: 需要在视图名前添加 **redirect:** 前缀。
 - 浏览器发起新请求, URL 地址栏改变。
 - 两次请求之间数据不共享, Model 数据无法直接传递。
 - 适用于跨控制器跳转或防止表单重复提交 (PRG 模式: Post-Redirect-Get)。
- ModelAndView 中的 Model 对象默认作用域为 **request** (请求作用域)。[p.28]

视图层解决方案 (View Layer Solutions)

- Spring MVC 支持多种 View 层解决方案: [p.25, 30]
 - **JSP (Java Server Pages)**: Spring MVC 默认视图技术; 语法标签化程度低, 前端后端耦合紧密; 不支持静态原型预览。
 - **FreeMarker**: 老版本推荐使用的模板引擎; 纯模板语法, 与 HTML 有一定分离; 在组件化和去耦合上有明显缺点。
 - **Thymeleaf**: Spring Boot 推荐, 当前主流; 核心特点是 **数据与 HTML 完全分离** (Separation of Data and HTML); 模板文件可直接在浏览器中以静态页面预览; 与 Spring MVC 深度集成。

- **Thymeleaf** 的核心优势: [p.30–32]
 - 静态原型 (Static Prototype): 模板即原型, 前端可直接在浏览器中查看效果。
 - 前后端并行开发: 前端工程师和后端工程师可以独立工作。
 - 自然模板 (Natural Template): 标签嵌入在标准 HTML5 属性中, 不破坏 HTML 结构。
- 模板语法对比: [p.31]
 - Velocity: `<p>${message}</p>`
 - FreeMarker: `<p>${message}</p>`
 - Thymeleaf: `<p th:text="${message}">Hello World!</p>`
- Thymeleaf 常用属性: `th:text` (文本替换)、`th:each` (循环)、`th:if/unless` (条件)、`th:href` (链接)、`th:src` (资源引用)。[p.31]

参数校验 (Parameter Validation)

@Valid 与 @Validation 注解对比 (Validation Annotation Comparison)

- @Validation 注解: [p.33]
 - Spring Framework 中提供的验证机制, 是 **JSR-303** 规范的一个变种 (Variant)。
 - 可以使用在类型 (Type)、方法 (Method) 和方法参数 (Parameter) 上。
 - 不能使用在类的成员属性上, 因此不支持嵌套验证。
 - Spring Framework 中默认使用 @Validation 注解进行参数校验。
- @Valid 注解: [p.33]
 - Hibernate 框架提供的验证机制, 完全符合 **JSR-303** 标准规范。
 - 比 @Validation 更强大, 可以使用在类的成员属性 (Member Field) 上。
 - 支持 **嵌套验证 (Nested Validation)**: 可验证对象内部的关联对象属性。
 - Spring Boot 同时集成了两种注解, 开发者可以自由选择使用。
- 核心区别: @Valid 支持成员属性和嵌套验证, @Validation 不支持。[p.33]

常用验证注解速查表 (Common Validation Annotations)

表 3: 常用 Bean Validation 注解速查 [p.34]

注解名	验证规则	说明
@NotNull	不能为 null	验证对象引用不为 null; 空字符串 "" 是允许的
@Null	必须为 null	验证对象引用必须为 null
@AssertTrue	必须为 true	验证 boolean/Boolean 类型必须为 true
@AssertFalse	必须为 false	验证 boolean/Boolean 类型必须为 false
@Digits(integer, fraction)	必须为数字, 指定位数范围	验证数字的整数位和小数位
@Max(value)	指定整数的最大值	值必须小于或等于指定值
@Min(value)	指定整数的最小值	值必须大于或等于指定值
@Length(min, max)	指定字符串长度范围	验证字符串长度在 [min, max] 之间

注解名	验证规则	说明
@Size(min, max)	指定集合/数组/字符串长度范围	比 @Length 更通用，适用于集合类型
@NotEmpty	不能为空	包括: null、空字符串 ("", length 为 0)、空集合 (size 为 0)
@NotBlank	不能为空 (trim 后)	包括: null、字符串首尾去空格后长度为 0
@Email	必须为 Email 格式	验证字符串是否符合 email 地址规范
@Pattern(regexp)	符合正则表达式	根据指定的正则表达式验证字符串格式

参数校验使用范例 (Usage Example)

- 在 Controller 方法参数前添加 @Valid 或 @Validation 注解触发校验。[p.35]
- 校验失败的参数后会紧跟一个 BindingResult 参数来接收校验错误信息，避免直接抛出异常。[p.35]
- 校验注解可以组合使用，如同时标注 @NotNull 和 @Size(min=2, max=30)。[p.35]
- 可自定义校验注解，通过实现 ConstraintValidator 接口来扩展业务相关的校验规则。[p.33]

Web 容器对象使用 (Web Container Objects)

使用 Web 容器对象的两种方式 (Coupled vs Decoupled)

- 在 Spring MVC 的 Controller 中使用 Web 容器对象 (HttpServletRequest, HttpServletResponse, HttpSession 等)，分为两种方式：[p.36]
- **耦合方式 (Coupled)**：[p.37]
 - 直接在 Controller 方法参数中声明并获取容器对象。
 - 代码与 Servlet API 紧密耦合，依赖于具体的 Servlet 容器实现。
 - 优点：直接使用熟悉的 Servlet API，灵活性高。
 - 缺点：不利于单元测试，更换容器环境可能需要修改代码。
- **非耦合方式 (Decoupled)**：[p.38]
 - 通过 Spring MVC 提供的抽象接口进行注入。
 - 降低对 Servlet API 的直接依赖，代码更具可移植性。
 - 有利于单元测试 (可轻松 Mock 容器对象)。
 - 例如使用 RequestContextHolder 在非 Controller 层获取请求信息。
- 推荐在开发中使用解耦合方式，仅在必要时使用耦合方式直接操作 Servlet API (如文件上传、Cookie 操作等特殊场景)。[p.38]

RESTful 风格与 @RestController (RESTful & @RestController)

- **REST (Representational State Transfer, 表现层状态转移)** 是一种 Web 服务架构风格，通过 URL 定位资源，通过 HTTP 方法操作资源。[p.14, 19, 25]
- **@RestController**：类级别注解，等同于 @Controller + @ResponseBody 的组合注解。[p.25]
- 标注 @RestController 的类中所有方法返回值均自动序列化为 JSON/XML 写入 HTTP 响应体，无需在每个方法上标注 @ResponseBody。[p.25]
- RESTful URL 设计规范：[p.19, 21]
 - GET /users — 查询用户列表

- GET /users/{id} — 查询单个用户
- POST /users — 新增用户
- PUT /users/{id} — 更新用户
- DELETE /users/{id} — 删除用户
- @PathVariable 配合 RESTful URL 提取资源 ID。[p.21]

拦截器 (Interceptors)

拦截器概述 (Overview)

- Spring MVC 拦截器 (Interceptor) 类似于 Servlet 技术中的过滤器 (Filter)，用于对请求进行前置 (Pre-handle) 和后置 (Post-handle) 的过滤处理。[p.41]
- 实现系统的 **plug-in (插件)** 功能，达到按业务功能部分 (Business Concern) 和非业务功能部分 (Cross-cutting Concern) 解耦的目的。[p.41]
- Spring MVC 拦截器的实现机制基于 **Spring AOP (面向切面编程)**，与 Servlet 中的过滤器及其他 Web 框架的拦截器实现机制不同。[p.41]
- 核心回调方法：[p.41–43]
 - preHandle(): Controller 执行前调用；返回 true 放行，返回 false 拦截。
 - postHandle(): Controller 执行后、视图渲染前调用；可修改 ModelAndView。
 - afterCompletion(): 视图渲染完成后调用；通常用于资源清理。

拦截器与过滤器对比 (Interceptor vs Filter)

表 4: 拦截器 (Interceptor) 与过滤器 (Filter) 对比 [p.41–42]

对比维度	拦截器 (Interceptor)	过滤器 (Filter)
实现机制	基于 Spring AOP (面向切面编程) 实现，由 Spring IoC 容器管理	基于 Java Servlet 规范实现，由 Servlet 容器管理
作用范围	只拦截经过 DispatcherServlet 的 Controller 请求 (不包含静态资源)	对所有进入 Web 容器的请求生效 (含静态资源)
控制粒度	更细粒度：可获取被拦截方法上下文 (Handler, Method, 参数等)	只能访问 Request/Response 对象
依赖注入	支持 Spring DI，可直接注入其他 Bean	不支持 Spring 依赖注入 (或需额外配置)
生命周期	由 Spring IoC 容器管理，随容器启动/销毁	由 Servlet 容器管理，随 Web 应用启动/销毁
适用场景	权限验证、日志记录、性能监控等业务相关拦截	字符编码、CORS、安全过滤等通用 Web 处理

拦截器典型应用场景 (Typical Use Cases)

- **登录权限检查 (Authentication Interceptor)**: 拦截未登录请求，重定向到登录页面；在 preHandle() 中检查 Session 中是否存在登录用户信息。[p.16]

- **日志记录 (Logging Interceptor)**: 记录每个请求的 URL、请求参数、响应时间、操作用户等信息。[p.44]
- **性能监控 (Performance Monitoring)**: 在 `preHandle()` 记录开始时间, 在 `afterCompletion()` 中计算执行耗时。[p.44]
- **结合 Logback 实现用户流量监控 (Traffic Monitoring)**: 通过拦截器收集请求数据, 使用 Logback 等日志框架进行统计分析。[p.44]
- **跨域处理 (CORS)**: 在拦截器中对响应头进行统一设置, 处理跨域请求。[p.42]

全局异常处理 (Global Exception Handling)

注解与机制 (Annotations & Mechanism)

- **@ExceptionHandler**: [p.45]
 - 方法级别注解, 用于处理特定类型的异常。
 - 可定义在 Controller 内部, 仅处理当前 Controller 抛出的异常。
 - 方法参数可包含异常对象 (`Exception e`)、`HttpServletRequest`、`HttpServletResponse` 等。
- **@ControllerAdvice**: [p.45]
 - 类级别注解, 定义全局异常处理器 (Global Exception Handler)。
 - 作用于所有 Controller (或通过属性指定特定的 Controller 包)。
 - 内部结合 `@ExceptionHandler` 使用, 集中处理所有 Controller 的异常。
 - 可配合 `@InitBinder` 和 `@ModelAttribute` 实现全局数据绑定和模型初始化。
- **@RestControllerAdvice**: `@ControllerAdvice` + `@ResponseBody` 的组合, 适用于 RESTful API 的全局异常处理, 返回 JSON 格式的错误响应。[p.45]

全局异常处理优势 (Advantages)

- 避免在每个 Controller 中重复编写 try-catch 逻辑, 实现异常处理的统一管理。[p.45]
- 保证一致的错误响应格式 (Uniform Error Response Format), 提升 API 的可用性。[p.45]
- 可以对不同类型的异常 (如参数校验异常、业务异常、系统异常) 定义不同的处理策略。[p.45]
- 结合自定义异常类和 HTTP 状态码注解 (`@ResponseStatus`), 实现精确的异常响应。[p.45]

文件上传 (File Upload)

- Spring MVC 支持文件上传功能, 基于 Apache Commons FileUpload 或 Servlet 3.0+ 的 Part API。
- 配置 **MultipartResolver (多部件解析器)**: 需在 Spring 容器中注册一个 `CommonsMultipartResolver` 或 `StandardServletMultipartResolver` 的 Bean。
- Controller 中使用 `@RequestParam("file") MultipartFile file` 接收上传的文件对象。
- **MultipartFile** 接口提供的主要方法:
 - `getOriginalFilename()`: 获取上传文件的原始文件名。
 - `getSize()`: 获取文件大小 (字节)。
 - `getContentType()`: 获取文件 MIME 类型。
 - `transferTo(File dest)`: 将上传的文件保存到目标位置。
 - `getBytes()`: 获取文件的字节数组。
- 限制文件上传大小: 通过 MultipartResolver 的 `maxUploadSize` 属性设置。
- 多文件上传: 可使用 `@RequestParam("files") MultipartFile[] files` 或 `List<MultipartFile>`。

HelloWorld 最小化配置回顾 (HelloWorld Recap)

- Maven 依赖: `spring-webmvc`。[p.11]
- `web.xml` 配置: 注册 `DispatcherServlet`, 指定 Spring 配置文件路径 (`contextConfigLocation`)。[p.12]
- `applicationContext.xml` 配置: 开启组件扫描 (`<context:component-scan>`)、配置视图解析器 (`InternalResourceViewResolver`)、开启 MVC 注解驱动 (`<mvc:annotation-driven>`)。[p.13]
- Controller 编写三要素: [p.14]
 - `@Controller` — 声明该类为 Spring MVC 控制器。
 - `@GetMapping("/hello")` — 映射 GET 请求到方法。
 - `@ResponseBody` — 直接返回字符串到 HTTP 响应体。
- 运行访问: 启动 Tomcat 后访问 `http://localhost:8080/hello`。[p.14]

登录功能实现示例 (Login Implementation)

- Spring MVC 实现登录功能的典型流程: [p.16]
 1. 用户通过表单提交用户名和密码 (POST 请求)。
 2. Controller 通过 `@RequestParam` 或 `JavaBean` 接收登录参数。
 3. Controller 调用 Service 层进行身份验证 (查询数据库或认证服务)。
 4. 验证成功: 将用户信息存入 `HttpSession`, 返回成功页面或 JSON 响应。
 5. 验证失败: 返回错误信息, 跳转回登录页面。
- 配合拦截器 (`AuthenticationInterceptor`) 实现登录状态检查: 在 `preHandle()` 中检查 Session 中是否存在用户信息, 若不存在则重定向到登录页面。[p.16, 41]

最佳实践 (Best Practices)

- 分层职责清晰: Controller 层只负责参数接收、校验和视图跳转, 不包含业务逻辑; 业务逻辑封装在 Service 层。[p.46]
- 异常处理统一化: 使用 `@ControllerAdvice` + `@ExceptionHandler` 集中处理异常, 避免分散的 `try-catch`。[p.45–46]
- 参数校验前置: 在 Controller 层使用 `@Valid`/`@Validation` 完成基础校验, Service 层完成业务校验。[p.46]
- 非耦合使用容器对象: 优先使用 Spring MVC 抽象接口, 减少对 Servlet API 的直接依赖, 方便单元测试。[p.38, 46]
- 响应格式统一: RESTful API 统一返回 JSON 格式, 包含状态码 (code)、消息 (message)、数据 (data) 三段式结构。[p.46]
- 拦截器合理使用: 非业务功能 (日志、权限、监控) 通过拦截器实现, 保持 Controller 方法简洁。[p.46]
- Controller 单例与线程安全: Spring MVC 的 Controller 默认为单例 (Singleton), 不要在 Controller 中定义可变实例变量; 使用局部变量、方法参数或 `ThreadLocal`。[p.46]

本章小结 (Chapter Summary)

Spring 框架与 Spring MVC 简介

- Spring Framework 是提供全面基础设施支持的 Java 平台, Spring MVC 是其轻量级 Web 表示层模块, 基于 IoC 容器运行。[p.47]
- Spring 5.x 要求 JDK 8 / Servlet 3.1, Spring 6.x 要求 JDK 17 / Servlet 5.0。[p.8, 47]

架构和核心原理

- DispatcherServlet (前端控制器) 是 Spring MVC 核心, 负责协调各组件完成请求处理。[p.47]
- 完整流程: DispatcherServlet → HandlerMapping (查映射) → Controller (处理) → ModelAndView (模型 + 视图) → ViewResolver (解析视图) → View (渲染) → Response (响应)。[p.10, 47]

使用详解核心技术点

- URL 映射: 基于方法的映射 (@RequestMapping / @GetMapping / @PostMapping / @PutMapping / @DeleteMapping), 粒度更细。[p.19, 47]
- 请求参数获取: @RequestParam (单个参数)、@RequestBody (JSON/JavaBean)、@PathVariable (路径变量)、@RequestHeader (请求头)。[p.21, 47]
- 响应处理: @ResponseBody (JSON 响应)、@RestController (组合注解)、ModelAndView (页面跳转 + 数据)。[p.25–28, 47]
- 数据容器: ModelMap < Model < ModelAndView (能力递增)。[p.29, 47]
- 视图解决方案: JSP (默认) → FreeMarker (过渡) → Thymeleaf (推荐, 数据与 HTML 分离)。[p.25, 30, 47]
- 参数校验: @Valid (Hibernate, JSR-303 标准, 支持嵌套验证) vs @Validation (Spring, 不支持成员属性)。[p.33, 47]
- Web 容器对象: 耦合方式 (直接声明) vs 非耦合方式 (抽象注入)。[p.36, 47]

扩展学习

- 拦截器 (Interceptor): 基于 Spring AOP 的请求前置/后置处理; 与 Servlet Filter 的区别 (实现机制/作用范围/控制粒度/依赖注入/适用场景)。[p.41–44, 47]
- 全局异常处理: @ControllerAdvice + @ExceptionHandler 实现统一的异常管理。[p.45, 47]
- 文件上传: MultipartFile + MultipartResolver, 支持单文件和多文件上传。
- RESTful API: @RestController + @PathVariable 实现资源驱动的 Web 服务设计。
- 最佳实践: 分层清晰、异常统一处理、参数校验前置、响应格式统一、Controller 线程安全等。[p.46–47]